

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessor - Microcontroller

Lab Report - CO3010 - CC02

Lab 5

Advisor(s): Phan Văn Sỹ

Student(s): Lương Đức Hiếu 2352324

HO CHI MINH CITY, NOVEMBER 2025



Contents

1	Overall	3
2	Proteus Schematic	3
3	Analyzing code	3
3.1	Global variables	3
3.2	UART interrupt handler	4
3.3	Command parser FSM	4
3.3.1	Sketch	6
3.3.2	Example parsing	6
3.4	Uart communication FSM	7
3.4.1	Sketch	8
3.4.2	State 0: IDLE (Wait for Command)	8
3.4.3	State 1: Read and send ADC	9
3.4.4	Stage 2: Wait OK + timeout	9
3.5	Main function	10
3.5.1	main.h	10
3.5.2	main.c	10


```
3 ADC_HandleTypeDef hadc1;
4 UART_HandleTypeDef huart2;
5
6 uint8_t temp = 0;
7 uint8_t buffer[MAX_BUFFER_SIZE];
8 uint8_t index_buffer = 0;
9 uint8_t buffer_flag = 0;
10 uint32_t ADC_value = 0;
11 char str[50];
12
13 uint8_t command_flag = 0;
14 char command_data[MAX_BUFFER_SIZE];
```

Purpose of each variable:

- temp: Receives single character from UART interrupt
- buffer[]: Circular buffer for incoming characters
- bufferflag: Signals main loop that new data arrived
- commandflag: Indicates complete command is ready
- commanddata[]: Stores parsed command string (e.g., "RST", "OK")

3.2 UART interrupt handler

```
1 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
2     if (huart->Instance == USART2) {
3         buffer[index_buffer++] = temp;
4         if (index_buffer == MAX_BUFFER_SIZE)
5             index_buffer = 0;
6         buffer_flag = 1;
7         HAL_UART_Receive_IT(&huart2, &temp, 1);
8     }
9 }
```

3.3 Command parser FSM



```
1 void command_parser_fsm(void) {
2     static uint8_t state = 0;
3     static uint8_t cmd_index = 0;
4
5     for (int i = 0; i < index_buffer; i++) {
6         uint8_t c = buffer[i];
7
8         switch (state) {
9             case 0: // Wait for '!'
10                if (c == '!') {
11                    state = 1;
12                    cmd_index = 0;
13                    memset(command_data, 0, MAX_BUFFER_SIZE);
14                }
15                break;
16
17             case 1: // Collect command characters
18                if (c == '#') {
19                    command_data[cmd_index] = '\0';
20                    command_flag = 1;
21                    state = 0;
22                } else {
23                    command_data[cmd_index++] = c;
24                    if (cmd_index >= MAX_BUFFER_SIZE - 1) {
25                        state = 0; // Reset on overflow
26                    }
27                }
28                break;
29            }
30        }
31        index_buffer = 0; // Clear buffer after processing
32    }
```

3.3.1 Sketch

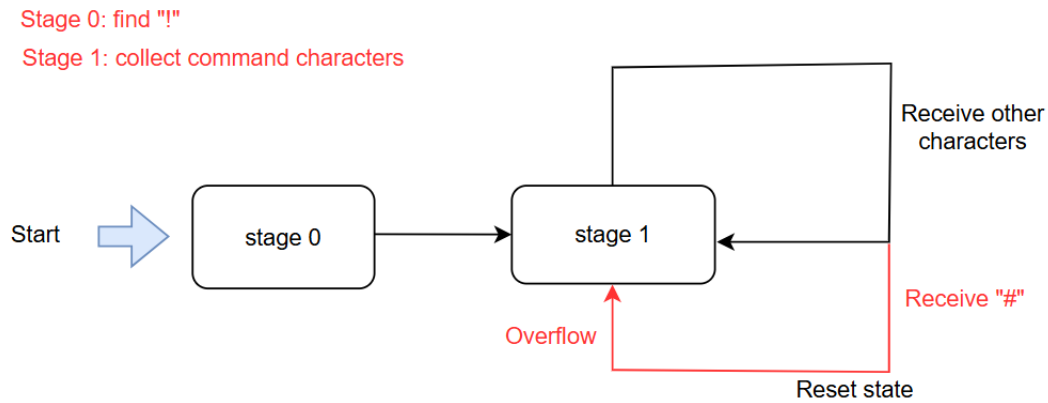


Figure 3.1: FSM Sketch

3.3.2 Example parsing

Input: !RST#

Position	Character	State	Action
0	!	0 → 1	Start collecting
1	R	1	command_data[0] = 'R'
2	S	1	command_data[1] = 'S'
3	T	1	command_data[2] = 'T'
4	#	1 → 0	command_data = "RST" command_flag = 1

Input: !OK#

Result: command_data = "OK", command_flag = 1



3.4 Uart communication FSM

```
1 void uart_communication_fsm(void) {  
2     static uint8_t comm_state = 0;  
3     static uint32_t last_send_time = 0;  
4     static uint32_t last_adc_value = 0;  
5  
6     switch (comm_state) {  
7         case 0: /* IDLE */  
8         case 1: /* READ & SEND ADC */  
9         case 2: /* WAIT OK + TIMEOUT */  
10    }  
11 }
```

3.4.1 Sketch

Stage 0: find "RST"

Stage 1: find "OK"/ timeout

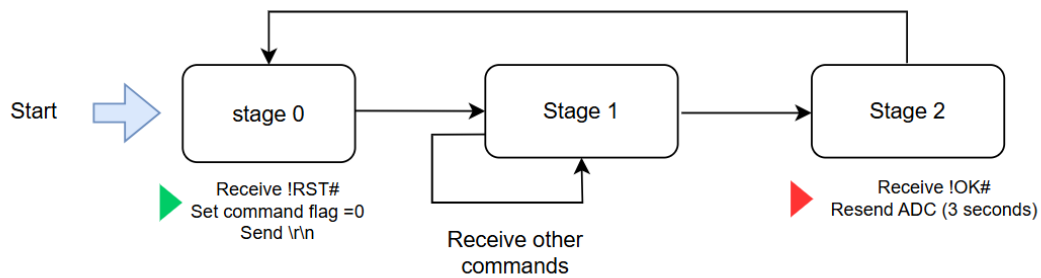


Figure 3.2: FSM Sketch

3.4.2 State 0: IDLE (Wait for Command)

```
1 case 0:
2     if (command_flag == 1) {
3         command_flag = 0;
4         if (strcmp(command_data, "RST") == 0) {
5             HAL_UART_Transmit(&huart2, (uint8_t *)"\r\n", 2, 1000)
6             ;
7             comm_state = 1;
8             last_send_time = HAL_GetTick();
9         }
10    }
11    break;
```

- Check if command received (`command_flag == 1`)
- Compare command with "RST"
- Send `\r\n` for clean formatting
- Transition to State 1
- Record timestamp

3.4.3 State 1: Read and send ADC

```
1 case 1:
2     HAL_ADC_Start(&hadc1);
3     if (HAL_ADC_PollForConversion(&hadc1, 100) == HAL_OK) {
4         ADC_value = HAL_ADC_GetValue(&hadc1);
5         last_adc_value = ADC_value;
6     }
7     HAL_ADC_Stop(&hadc1);
8     sprintf(str, "$!ADC=%lu#\r\n", ADC_value);
9     HAL_UART_Transmit(&huart2, (uint8_t *)str, strlen(str), 1000);
10    comm_state = 2;
11    last_send_time = HAL_GetTick();
12    break;
```

- Start ADC: Begin analog-to-digital conversion
- Poll for completion: Wait up to 100ms
- Read value: Get 12-bit result (0-4095)
- Save value: Store in last_adc_value for timeout resend
- Format string: Create "\$!ADC=xxxx#\r\n"
- Transmit: Send via UART (timeout 1000ms)
- Record time: Save current tick for timeout calculation
- Change state: Move to State 2

3.4.4 Stage 2: Wait OK + timeout

```
1 case 2:
2     if (command_flag == 1) {
3         command_flag = 0;
4         if (strcmp(command_data, "OK") == 0) {
5             comm_state = 0; // Return to idle
6         }
7     }
```

```
8
9  if ((HAL_GetTick() - last_send_time) > 3000) {
10      sprintf(str, "$!ADC=%lu#\r\n", last_adc_value);
11      HAL_UART_Transmit(&huart2, (uint8_t *)str, strlen(str),
12                          1000);
13      last_send_time = HAL_GetTick();
14  }
15  break;
```

Receive !OK#

- Clear command flag
- Return to State 0 (IDLE)
- Ready for next !RST# command

Timeout (3 seconds)

- Calculate: $\text{current_time} - \text{last_send_time} > 3000\text{ms}$
- Resend the same ADC value (from last_adc_value)
- Reset timer
- Stay in State 2

3.5 Main function

3.5.1 main.h

```
1 #define ADC1_IN0_Pin  GPIO_PIN_13
2 #define ADC1_IN0_GPIO_Port  GPIOC
3 #define LED_RED_Pin  GPIO_PIN_5
4 #define LED_RED_GPIO_Port  GPIOA
```

3.5.2 main.c

```
1 int main(void) {
2     HAL_Init();
3     SystemClock_Config();
```



```
4  MX_GPIO_Init();
5  MX_ADC1_Init();
6  MX_USART2_UART_Init();
7
8  HAL_UART_Receive_IT(&huart2, &temp, 1);
9  HAL_ADC_Start(&hadc1);
10
11 while (1) {
12     HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
13     HAL_Delay(500);
14
15     if (buffer_flag == 1) {
16         command_parser_fsm();
17         buffer_flag = 0;
18     }
19     uart_communication_fsm();
20 }
21 }
```